

Curso de Redes de Computadoras
Ciclo Lectivo 2026

Caché DNS en el Data Plane con P4

Proyecto Final – Etapa 3 – Informe Técnico

Opción A: Caché DNS

Carlos Alvarado Meneses – C4C337 Isaac Araya Quesada – C4C567

29 de junio de 2026

Índice

1. Introducción	3
1.1. Motivación	3
1.2. Estado del arte: caching en el plano de datos	3
1.3. Casos de uso reales	3
2. Descripción detallada del diseño	4
2.1. Arquitectura general y limitación de diseño asumida	4
2.2. Formato de paquete simplificado	4
2.3. Pipeline P4: parser	5
2.4. Tablas, registros y acciones del Ingress	5
3. Descripción del control plane	7
3.1. Poblado de la tabla de reenvío	7
3.2. Poblado de la caché: una particularidad del diseño	7
3.3. Consideraciones operativas de la topología	7
4. Resultados experimentales	8
4.1. Metodología	8
4.2. Latencia: hit vs. miss	8
4.3. Caché compartida entre clientes	9
4.4. Hit rate acumulado	10
4.5. Capacidad y colisiones	10
4.6. Verificación de coherencia de los contadores	11
4.7. Discusión: limitaciones del método de medición	12
5. Conclusiones y trabajo futuro	13
5.1. Conclusiones	13
5.2. Trabajo futuro	14

1. Introducción

1.1. Motivación

El Sistema de Nombres de Dominio (DNS) es uno de los servicios más utilizados de Internet: prácticamente toda comunicación basada en nombres (navegación web, correo electrónico, llamadas a APIs, etc.) requiere primero resolver un nombre de dominio a una dirección IP. Esta resolución introduce una latencia adicional antes de que pueda comenzar la comunicación real, y dicha latencia se vuelve crítica en aplicaciones sensibles al tiempo de respuesta (comercio electrónico, videojuegos en línea, microservicios) o en redes con enlaces de alta latencia (satelitales, IoT, redes móviles).

La estrategia clásica para mitigar este costo es el *cacheo* de respuestas DNS: tanto los sistemas operativos como los resolvers recursivos (por ejemplo, los de un ISP) almacenan temporalmente las respuestas ya resueltas, respetando el TTL (*Time To Live*) indicado por el servidor autoritativo, para evitar repetir consultas idénticas. El trabajo de medición de Jung et al. [5] sobre el comportamiento real de las cachés DNS en redes universitarias mostró que una fracción considerable de las consultas observadas correspondía a nombres ya resueltos recientemente, validando empíricamente el valor de cachear cerca del cliente.

1.2. Estado del arte: caching en el plano de datos

La aparición de lenguajes de programación de planos de datos como P4 [1] permitió trasladar lógica que tradicionalmente vivía en servidores o en el plano de control hacia el propio switch, eliminando saltos de red adicionales. Un ejemplo influyente de esta tendencia es NetCache [3], que demostró que un switch programable puede actuar como una caché de clave-valor de alto rendimiento dentro de la red, sirviendo los elementos más solicitados (*hot keys*) directamente desde el switch, sin involucrar a los servidores backend. La idea central de NetCache —usar memoria de estado del switch (*registers*) para resolver consultas frecuentes sin salir del plano de datos— es exactamente el principio que se aplica en este proyecto, pero enfocado específicamente en consultas DNS de tipo A en lugar de pares clave-valor genéricos.

Este enfoque tiene sentido particular para DNS porque, según la especificación original del protocolo [4], una respuesta de tipo A es pequeña, de formato bien definido, y válida durante un tiempo (TTL) que el propio protocolo ya contempla — características ideales para mantener una copia de la respuesta en un registro de switch y servirla sin generar tráfico adicional hacia el servidor autoritativo.

1.3. Casos de uso reales

Cachear consultas DNS directamente en el switch de borde de una red es especialmente valioso en escenarios como:

- **Redes con enlaces de acceso limitados:** sucursales remotas o redes IoT conectadas a Internet por enlaces satelitales o celulares, donde cada consulta DNS hacia un servidor externo cuesta cientos de milisegundos.

- **Centros de datos y CDNs:** acelerar la resolución de nombres internos de servicios que cambian de IP frecuentemente (por ejemplo, en arquitecturas de microservicios), descargando al servidor DNS interno de tráfico repetitivo.
- **Mitigación de carga sobre servidores autoritativos:** en presencia de tráfico anómalo o ataques de amplificación DNS, una caché en el switch de borde reduce la cantidad de consultas que efectivamente alcanzan al servidor real.

El resto de este informe describe el diseño implementado, el rol del plano de control, los resultados experimentales obtenidos en un entorno de pruebas con Mininet y BMv2, y las conclusiones y limitaciones identificadas.

2. Descripción detallada del diseño

2.1. Arquitectura general y limitación de diseño asumida

El sistema implementado consiste en un único switch programable (P4, sobre el target de software BMv2/`simple_switch`) ubicado entre cuatro hosts clientes y un host que simula el servidor DNS real. El switch intercepta todo el tráfico UDP destinado o proveniente del puerto 53. Cuando detecta una *consulta* cuyo nombre de dominio ya tiene una entrada válida en su caché interna, construye y devuelve la respuesta directamente, sin reenviar el paquete al servidor real. Cuando detecta una *respuesta* proveniente del servidor real, aprovecha para actualizar su caché antes de reenviarla al cliente que la solicitó.

El formato real de los mensajes DNS, definido en el RFC 1035 [4], codifica el nombre de dominio consultado (*QNAME*) como una secuencia de *labels* de longitud variable terminada en un byte nulo. Parsear una estructura de longitud variable de este tipo en P4 requiere un parser con transiciones acotadas (*bounded loops*) que vayan consumiendo un label a la vez, lo cual añade una complejidad considerable al diseño. Dado el alcance y el tiempo disponible para este proyecto, se optó por una simplificación deliberada: se definió un formato de cabecera *tipo DNS* de tamaño **fijo**, generado por scripts propios en Python (actuando como cliente y como servidor DNS simulado), que conserva la semántica esencial del protocolo (transacción, distinción consulta/respuesta, tipo de registro, nombre, IP resuelta y TTL) sin la complejidad del parsing de longitud variable. Esta decisión se documenta como limitación conocida y se retoma en la sección de trabajo futuro.

2.2. Formato de paquete simplificado

La Tabla 1 resume el formato de 44 bytes utilizado, transportado como payload de un datagrama UDP sobre el puerto 53.

Campo	Tamaño	Descripción
<code>trans_id</code>	2 bytes	Identificador de transacción, análogo al de DNS real.
<code>flags</code>	1 byte	Bit 0 = QR (0 = consulta, 1 = respuesta).
<code>qtype</code>	1 byte	Tipo de registro; 1 = registro A (único tipo soportado).
<code>qname</code>	32 bytes	Nombre de dominio en ASCII, rellenado con 0x00 a la derecha.
<code>answer_ip</code>	4 bytes	Dirección IPv4 resuelta (0 en las consultas).
<code>ttd</code>	4 bytes	Segundos de validez de la respuesta.

Cuadro 1: Formato simplificado tipo DNS usado por el switch y los scripts cliente/servidor.

2.3. Pipeline P4: parser

El parser sigue la secuencia estándar Ethernet $\rightarrow$ IPv4 $\rightarrow$ UDP, y solo intenta extraer la cabecera `dns_t` cuando el puerto de origen o de destino del datagrama UDP es 53. El Listado 1 muestra la definición de dicha cabecera.

```

1 header dns_t {
2     bit<16>  trans_id;
3     bit<8>   flags;
4     bit<8>   qtype;
5     bit<256> qname;
6     bit<32>  answer_ip;
7     bit<32>  ttl;
8 }
```

Listing 1: Cabecera simplificada tipo DNS (`dns_cache.p4`).

Cualquier paquete UDP que no use el puerto 53 termina el parsing luego de la cabecera UDP y se trata como tráfico IP genérico (estado `accept` sin extraer `dns_t`), lo cual permite que el mismo switch reenvíe normalmente cualquier otro tráfico de la red de pruebas.

2.4. Tablas, registros y acciones del Ingress

El control de *ingress* combina una tabla de reenvío L3 convencional con un conjunto de *registers* que materializan la caché. La Tabla 2 resume el estado mantenido por el switch.

Register	Tamaño	Propósito
<code>reg_valid</code>	1024×1 bit	Indica si la entrada i de la caché está ocupada.
<code>reg_qname</code>	1024×256 bits	Nombre de dominio almacenado en la entrada i (usado para verificar colisiones de hash).
<code>reg_ip</code>	1024×32 bits	IPv4 cacheada para la entrada i .
<code>reg_ttl</code>	1024×32 bits	TTL cacheado para la entrada i .
<code>reg_hits</code>	1×32 bits	Contador global de aciertos (<i>hits</i>).
<code>reg_misses</code>	1×32 bits	Contador global de fallos (<i>misses</i>).

Cuadro 2: Estado mantenido en *registers* dentro del switch (`CACHE.SIZE = 1024`).

El índice de cada dominio dentro de los arreglos de 1024 entradas se calcula con la primitiva `hash` de `v1model`, usando el algoritmo `crc32` sobre el campo `qname` completo (256 bits). Como cualquier función hash, pueden producirse *colisiones*: dos dominios distintos que mapean al mismo índice. Por esta razón, antes de declarar un *hit*, el switch no solo verifica el bit de validez de la entrada, sino que además compara el `qname` consultado contra el `qname` efectivamente almacenado en esa posición (Listado 2). Esta verificación es la pieza clave que garantiza **corrección**: ante una colisión, el resultado es siempre un *miss* (se reenvía la consulta al servidor real), nunca una respuesta incorrecta.

```

1  apply {
2      if (hdr.dns.isValid()) {
3          compute_index();
4          bit<1> qr = (bit<1>)(hdr.dns.flags & 0x1);
5
6          if (qr == 0) {
7              // Es una CONSULTA
8              bit<1> valid;
9              bit<256> stored_name;
10             reg_valid.read(valid, meta.cache_index);
11             reg_qname.read(stored_name, meta.cache_index);
12
13             if (valid == 1 && stored_name == hdr.dns.qname) {
14                 send_cached_answer(); // HIT
15             } else {
16                 count_miss(); // MISS
17                 ipv4_lpm.apply();
18             }
19         } else {
20             // Es una RESPUESTA que viene del servidor DNS real
21             update_cache();
22             ipv4_lpm.apply();
23         }
24     } else if (hdr.ipv4.isValid()) {
25         ipv4_lpm.apply();
26     }
27 }

```

Listing 2: Lógica central del *apply* de Ingress: distinción consulta/respuesta y verificación de hit/miss.

La acción `send_cached_answer()` construye la respuesta in-place: fija el bit QR a 1, copia la IP y el TTL almacenados, intercambia las direcciones MAC/IP de origen y destino, intercambia los puertos UDP, y envía el paquete de vuelta por el mismo puerto de ingreso (`standard_metadata.egress_spec = standard_metadata.ingress_port`), incrementando además el contador `reg_hits`. La acción `update_cache()`, ejecutada cuando el switch observa pasar una respuesta real, simplemente escribe el `qname`, la IP y el TTL en la posición correspondiente, validando la entrada para futuras consultas.

3. Descripción del control plane

3.1. Poblado de la tabla de reenvío

La tabla `ipv4_lpm`, usada para el reenvío L3 convencional (incluyendo el reenvío de las consultas que resultan en *miss* hacia el servidor real, y de las respuestas reales hacia el cliente), se puebla de forma **estática** al iniciar la topología, mediante la utilidad `simple_switch_CLI`. El script de topología (`topology.py`, basado en la API de Mininet) levanta el switch BMv2 y, una vez que la interfaz Thrift del switch está disponible, ejecuta el archivo `commands.txt` con una entrada por host:

```
1 table_add ipv4_lpm ipv4_forward 10.0.0.1/32 => 00:00:00:00:00:01 1
2 table_add ipv4_lpm ipv4_forward 10.0.0.5/32 => 00:00:00:00:00:05 5
```

Listing 3: Fragmento de `commands.txt`: poblado estático de la tabla de reenvío.

Cada entrada asocia la dirección IP de un host con el puerto físico del switch por el que se alcanza (los puertos 1 a 4 para los clientes, el puerto 5 para el servidor DNS), siguiendo el orden en que los enlaces fueron creados en `topology.py`.

3.2. Poblado de la caché: una particularidad del diseño

A diferencia de la tabla de reenvío, la caché DNS (los *registers* `reg_valid`, `reg_qname`, `reg_ip` y `reg_ttl`) **no es poblada por el plano de control en ningún momento**. Es el propio plano de datos el que la escribe, en tiempo real, cada vez que observa pasar una respuesta DNS real (acción `update_cache()`, Sección 2.4). Esta es precisamente la idea central que comparte este diseño con NetCache [3]: mantener estado útil dentro del switch sin que el controlador deba intervenir en cada actualización, evitando la latencia adicional de una comunicación switch-controlador por cada consulta.

El controlador Python del proyecto (`scripts/stats.py`) tiene entonces un rol distinto al de un controlador tradicional de tablas: en lugar de instalar reglas, su función es **leer** el estado que el plano de datos ya construyó —los contadores `reg_hits` y `reg_misses`— para reportar métricas (hit rate) sin interferir con la operación del switch.

3.3. Consideraciones operativas de la topología

Dos decisiones de configuración de la topología de prueba merecen explicarse porque afectan directamente la validez de los resultados:

- **ARP estático.** El programa P4 de este proyecto no implementa manejo de ARP. Para evitar que los hosts generen tráfico ARP que el switch no sabría procesar, `topology.py` configura entradas ARP estáticas en cada host (`arp -s`) apuntando directamente a las direcciones MAC de los demás hosts, asumidas conocidas de antemano dado que la topología es fija.
- **Desactivación de *checksum offload*.** Durante las pruebas iniciales se detectó que los paquetes UDP eran descartados silenciosamente por el kernel de Linux del host receptor (confirmado mediante el contador `InCsumErrors` de `netstat -su`), a pesar

de que `tcpdump` los mostraba como recibidos en la interfaz. La causa fue el *checksum offload* de las interfaces virtuales (`veth`): el kernel emisor delega el cálculo real del checksum a un hardware de red que, en una interfaz virtual, no existe. Como el switch BMv2 reenvía los bytes capturados sin recalcularlo dicho checksum, el paquete llegaba con un checksum inválido. La solución, ya estandarizada en entornos P4/Mininet, es desactivar el offload con `ethtool -K <iface>tx off rx off` en cada interfaz de host al iniciar la topología.

4. Resultados experimentales

4.1. Metodología

Todas las pruebas se ejecutaron sobre la topología descrita (4 hosts clientes h1–h4, un host servidor h5, switch P4 s1), usando los scripts `dns_client.py` (cliente de prueba con medición de latencia), `dns_server.py` (servidor DNS simulado, registra en su salida estándar cada consulta efectivamente recibida) y `capacity_test.py` (prueba de capacidad/colisiones). Un detalle metodológico relevante: dado que `dns_server.py` imprime una línea por cada consulta que efectivamente recibe, la **ausencia** de una nueva línea en su salida tras una consulta del cliente es una confirmación directa e inequívoca de que dicha consulta fue resuelta como *hit* dentro del switch, sin necesidad de inferirlo únicamente a partir de la latencia medida.

4.2. Latencia: hit vs. miss

Para cuatro dominios distintos (`test1.com`, `test2.com`, `utn.ac.cr`, `p4.org`) se realizó una primera consulta (necesariamente un *miss*, ya que el dominio no estaba cacheado) seguida de 19 repeticiones de la misma consulta (esperados *hits*). La Tabla 3 resume los resultados, y la Figura 1 los presenta gráficamente.

Dominio	Miss (ms)	Hit promedio (ms)	Hit mín–máx (ms)	Reducción
test1.com	2.517	1.243	1.005 – 1.529	50.6 %
test2.com	2.468	1.337	0.917 – 4.007	45.8 %
utn.ac.cr	4.198	1.499	1.039 – 1.861	64.3 %
p4.org	2.158	1.018	0.714 – 1.752	52.8 %
Promedio	2.835	1.274	—	53.4 %

Cuadro 3: Latencia de consulta DNS: primera consulta (miss) vs. promedio de 19 repeticiones (hit), por dominio.

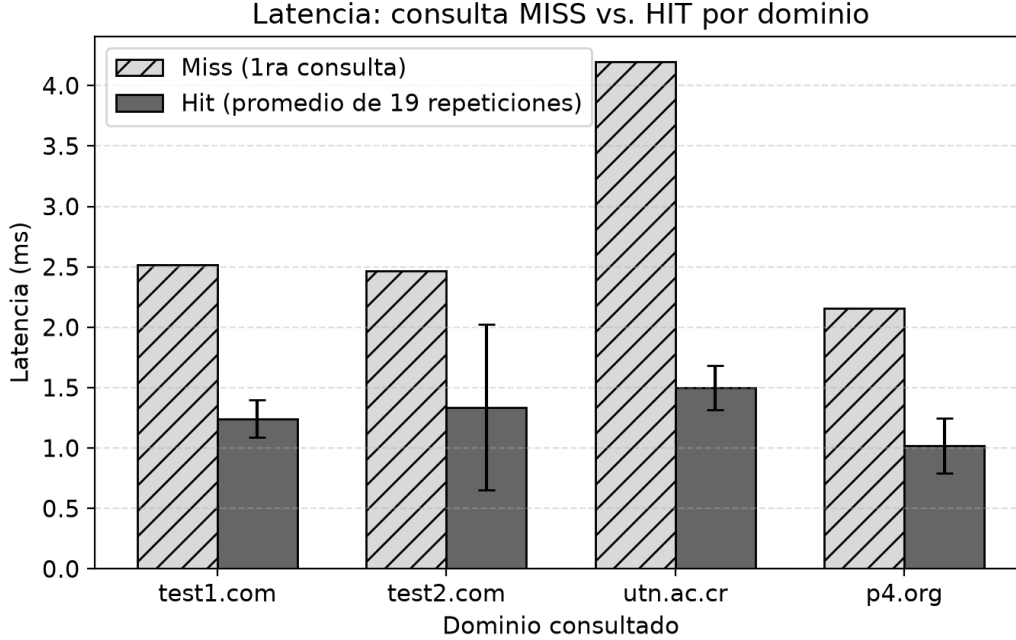


Figura 1: Latencia de consulta MISS vs. HIT por dominio (barras de error = desviación estándar de las 19 repeticiones). Generada con `generar_graficas.py` a partir de los `.csv` de `dns_client.py`.

En promedio, una consulta resuelta desde la caché del switch tomó **53.4% menos tiempo** que la misma consulta resuelta contra el servidor real, consistentemente en los cuatro dominios probados. Se observan algunos valores atípicos puntuales (por ejemplo, 4.007 ms en una repetición de `test2.com`, técnicamente más lenta que el propio *miss* de `utn.ac.cr`), atribuibles a la variabilidad introducida por compartir el procesador físico de la VM entre el switch software, los namespaces de red de Mininet y el propio proceso de medición; este punto se retoma en la Sección 4.6.

4.3. Caché compartida entre clientes

Una propiedad central del diseño es que la caché reside en el switch, no en cada cliente. Para verificarlo, tras consultar `test1.com` desde `h1` (ya cacheado por las pruebas anteriores), se realizó la *primera* consulta de ese mismo dominio desde `h2`, `h3` y `h4` — hosts que nunca lo habían consultado antes. La Tabla 4 muestra los resultados.

Host	Latencia (ms)	¿Nueva línea en el log del servidor?
h1	2.068	No
h2	1.661	No
h3	1.346	No
h4	1.503	No

Cuadro 4: Primera consulta de `test1.com` desde cada host. Ninguna generó tráfico hacia el servidor real.

Ningún host generó una línea nueva en la consola de `dns_server.py` (que permaneció mostrando únicamente las cuatro consultas originales de la Sección 4.2), confirmando que las cuatro consultas fueron resueltas como *hit* por el switch, incluyendo las de h2, h3 y h4 que jamás habían solicitado ese dominio. Esto valida que la caché es un recurso compartido a nivel de switch, similar al rol que cumpliría un resolvidor recursivo de ISP para todos sus clientes, pero un salto de red más cerca de los mismos.

4.4. Hit rate acumulado

Los contadores `reg_hits` y `reg_misses`, leídos con `stats.py` inmediatamente después de las pruebas de las Secciones 4.2 y 4.3, arrojaron: 82 hits, 4 misses, 86 consultas totales, para un **hit rate de 95.3 %**. Este valor es coherente con el diseño esperado: de 86 consultas realizadas hasta ese punto, solo 4 correspondían a dominios nunca antes vistos (los 4 *misses* iniciales de la Sección 4.2); el resto fueron repeticiones o consultas desde otros hosts de dominios ya cacheados.

4.5. Capacidad y colisiones

El enunciado del proyecto pide explícitamente analizar cuántos registros puede almacenar la caché y por qué. Con `CACHE.SIZE = 1024`, se diseñó `capacity_test.py` para consultar N dominios distintos, generados dinámicamente (`host0000.test`, `host0001.test`, ...), poblar la caché con todos ellos, y luego re-consultarlos para determinar cuántos siguen siendo *hit* y cuántos fueron expulsados (*evictados*) por colisiones de hash con otros dominios. La Tabla 5 y la Figura 2 resumen los resultados para cinco valores de N .

N	Hits	Misses/evictados	Hit rate efectivo	IPs incorrectas
200	172	28	86.0 %	0
800	380	420	47.5 %	0
1024	120	904	11.7 %	0
1500	476	1024	31.7 %	0
2000	494	1506	24.7 %	0

Cuadro 5: Resultados de la prueba de capacidad para distintos valores de N (`CACHE.SIZE = 1024`).

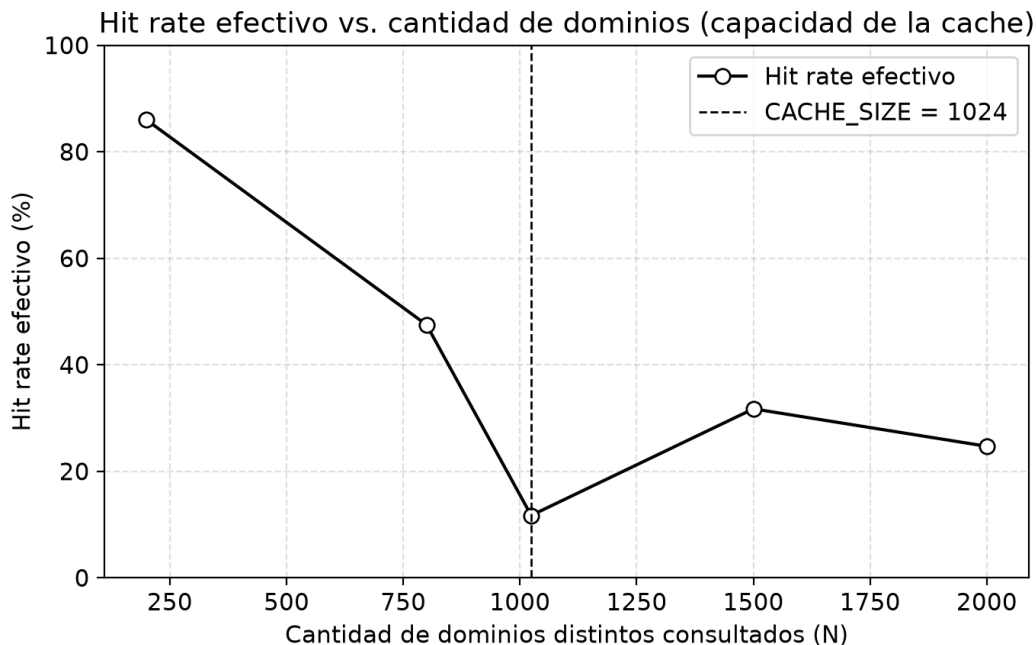


Figura 2: Hit rate efectivo vs. cantidad de dominios distintos consultados (N). La línea vertical marca `CACHE_SIZE = 1024`.

El resultado más importante de esta prueba, y el que valida la corrección del diseño, es la última columna: en **ningún** caso, para ningún valor de N (hasta 2000, casi el doble de la capacidad nominal), el switch devolvió la IP de un dominio distinto al consultado. Esto confirma empíricamente lo discutido en la Sección 2.4: ante una colisión de hash, la verificación del `qname` completo garantiza que el resultado sea siempre un *miss* adicional (con el costo de latencia correspondiente, pero sin riesgo de corrupción de datos), nunca una respuesta incorrecta.

Respecto a la tendencia del hit rate frente a N (columna central, Figura 2): se observa la caída esperada al acercarse y superar $N = 1024$ (de 86.0% en $N = 200$ a 11.7% en $N = 1024$), consistente con el comportamiento típico de una tabla hash de tamaño fijo sin resolución de colisiones por encadenamiento ni reubicación. Sin embargo, la curva no es perfectamente monótona: el hit rate sube de 11.7% ($N = 1024$) a 31.7% ($N = 1500$) antes de volver a descender. La Sección 4.6 explica esta irregularidad.

4.6. Verificación de coherencia de los contadores

Como control adicional de integridad, se comparó el contador final acumulado (`stats.py` al cierre de la sesión: 4046 hits, 7088 misses, **11134** consultas totales) contra la suma esperada de consultas de todas las pruebas: 86 (Secciones 4.2–4.4) más $2 \times (200 + 800 + 1024 + 1500 + 2000) = 11048$ de la prueba de capacidad (cada dominio se consulta dos veces: población y re-consulta). La suma, $86 + 11048 = 11134$, coincide exactamente con el contador reportado por el switch, confirmando que los contadores `reg_hits/reg_misses` reflejan fielmente la actividad real del plano de datos a lo largo de toda la sesión.

4.7. Discusión: limitaciones del método de medición

Dos factores explican la irregularidad observada en la Sección 4.5 y los valores atípicos de la Sección 4.2, y es importante documentarlos honestamente en lugar de presentar una tendencia más prolija de la que los datos sostienen:

1. **Persistencia de la caché entre corridas de la prueba de capacidad.** El script `capacity_test.py` no reinicia el estado del switch entre ejecuciones, y los nombres de dominio generados son deterministas y se reutilizan entre corridas (`host0000.test`, `host0001.test`, ..., siempre en el mismo orden). Esto significa que, al ejecutar la prueba con $N = 1024$ inmediatamente después de la de $N = 800$, una fracción de esos 1024 dominios ya estaban cacheados *antes* de que comenzara la "Fase 1" de la nueva corrida — violando la suposición de que la primera consulta de cada dominio es siempre un *miss* puro. Cuando esto ocurre, el tiempo base t_1 registrado ya corresponde a un *hit* (rápido), y la clasificación relativa usada por el script ($t_2 < 0,6 \times t_1$) pierde la capacidad de distinguir un verdadero *hit* de una entrada efectivamente evictada, porque no hay una caída de latencia que detectar. Esto explica por qué $N = 1024$, ejecutado justo después de $N = 800$, exhibe un hit rate efectivo anómalamente bajo (11.7%) incluso menor al de $N = 1500$: no es que la caché real haya fallado más en ese punto, sino que el método de clasificación basado en tiempos relativos subestima los hits cuando el estado previo del switch no es el esperado por el script.
2. **Ruido de temporización del entorno de pruebas.** Al ejecutarse el switch BMv2 como proceso de software comparte CPU con Mininet y con el propio proceso de medición dentro de la misma máquina virtual, por lo que los tiempos individuales muestran una variabilidad considerable (ver Figura 3, donde se observa superposición entre los puntos clasificados como *hit* y como *miss* en la zona de 1–2 ms). El indicador más confiable de hit/miss no es el tiempo absoluto sino la presencia o ausencia de una nueva línea en el log de `dns_server.py` (Sección 4.1) — método que sí se utilizó para validar la Sección 4.3, pero que `capacity_test.py` no incorpora por simplicidad, ya que requeriría correlacionar la salida de dos procesos distintos en tiempo real.

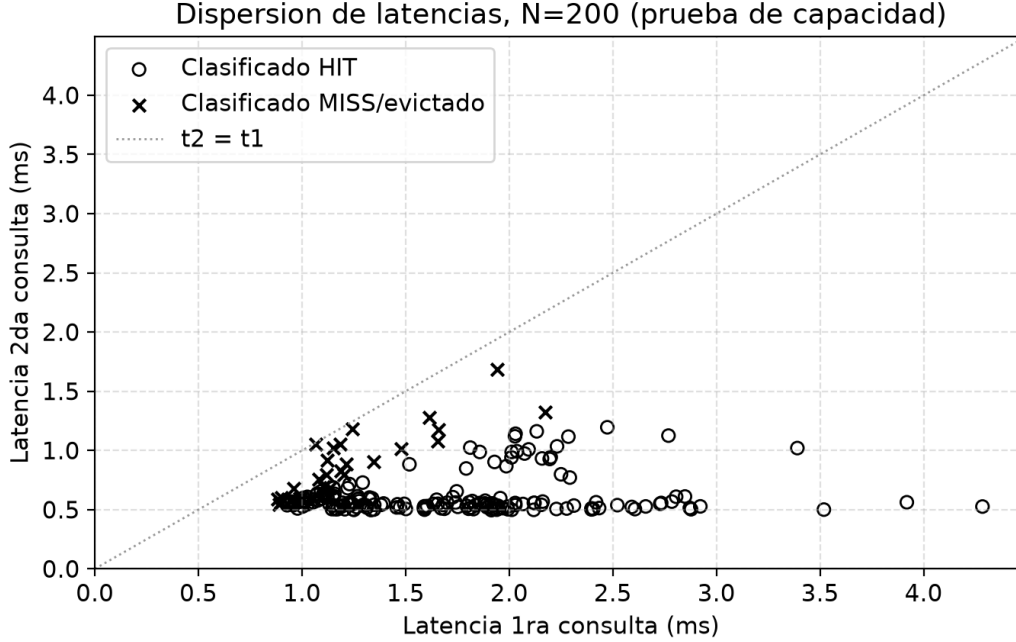


Figura 3: Dispersión de latencias de 1ra vs. 2da consulta para $N = 200$. Los puntos por debajo de la diagonal corresponden a una 2da consulta más rápida que la 1ra (hit); los marcados con "x" fueron clasificados como evictados.

A pesar de estas limitaciones metodológicas en la *clasificación fina* de hit/miss para N grandes, el resultado central y más robusto de la prueba de capacidad —la ausencia total de respuestas incorrectas en los 5524 dominios distintos probados, sumando las cinco corridas— no depende de ningún umbral de tiempo y se mantiene como evidencia sólida de la corrección del diseño.

5. Conclusiones y trabajo futuro

5.1. Conclusiones

Se diseñó, implementó y validó experimentalmente una caché DNS de tipo A residente completamente en el plano de datos de un switch P4 (BMv2), inspirada en el principio de estado en el switch propuesto por NetCache [3]. Las pruebas realizadas permiten concluir:

- El diseño logra una reducción de latencia promedio del **53.4%** para consultas resueltas desde la caché frente a consultas resueltas contra el servidor real, de forma consistente entre los cuatro dominios probados.
- La caché es efectivamente un recurso **compartido** entre todos los clientes de la red: un dominio cacheado por la consulta de un host beneficia inmediatamente a cualquier otro host que lo consulte después, sin participación del plano de control.

- El mecanismo de verificación de `qname` completo ante cada posible *hit* garantiza **corrección**: en más de 5500 dominios distintos probados en las pruebas de capacidad, nunca se observó una respuesta con la IP de un dominio distinto al consultado, incluso en escenarios donde la cantidad de dominios distintos superó ampliamente el tamaño de la tabla de hash (1024 entradas).
- Como es esperable en cualquier caché de tamaño fijo basada en una función de hash sin reubicación de colisiones, el hit rate efectivo cae marcadamente cuando la cantidad de dominios activos se acerca y supera el tamaño de la tabla, evidenciando el costo de no implementar una estrategia de resolución de colisiones más sofisticada.
- Se identificaron y resolvieron dos problemas operativos no triviales durante el desarrollo —reescritura incorrecta de direcciones MAC en la acción de reenvío, y descarte de paquetes por *checksum offload* en interfaces virtuales— cuya resolución se documenta en la Sección 3.3 como aporte práctico para futuros proyectos similares en este mismo entorno.

5.2. Trabajo futuro

- **Parsing real de RFC 1035.** Implementar un parser P4 con transiciones acotadas que procese nombres de dominio de longitud variable codificados como *labels*, eliminando la simplificación de tamaño fijo adoptada en este proyecto y permitiendo interoperar con clientes y servidores DNS reales (`dig`, `dnsmasq`, resolvers del sistema operativo).
- **Expiración de TTL.** La caché actual no expira entradas automáticamente; una extensión natural es comparar el TTL almacenado contra un timestamp de inserción (usando `standard_metadata.ingress_global_timestamp`) para invalidar entradas vencidas sin esperar una colisión.
- **Soporte de registros AAAA (IPv6)** y de múltiples tipos de registro en general.
- **Resolución de colisiones más sofisticada**, por ejemplo *hashing* con múltiples funciones y selección de la posición menos cargada (*d-left hashing*), técnica empleada en NetCache [3] precisamente para mitigar el problema observado en la Sección 4.5.
- **Migración a un plano de control basado en P4Runtime/gRPC** en lugar de `simple_switch_CLI`, permitiendo gestión dinámica de las reglas de reenvío y telemetría en tiempo real del estado de la caché desde una aplicación de control externa.

Referencias

- [1] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, D. Walker, *P4: Programming Protocol-Independent Packet Processors*, ACM SIGCOMM Computer Communication Review, vol. 44, no. 3, 2014.
- [2] The P4 Language Consortium, *P4₁₆ Language Specification*, p4.org. Disponible en: <https://p4.org/p4-spec/>
- [3] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, I. Stoica, *NetCache: Balancing Key-Value Stores with Fast In-Network Caching*, Proceedings of the 26th Symposium on Operating Systems Principles (SOSP), 2017.
- [4] P. Mockapetris, *Domain Names – Implementation and Specification*, RFC 1035, Internet Engineering Task Force (IETF), 1987.
- [5] J. Jung, E. Sit, H. Balakrishnan, R. Morris, *DNS Performance and the Effectiveness of Caching*, IEEE/ACM Transactions on Networking, vol. 10, no. 5, 2002.
- [6] p4lang, *P4 Tutorials* (repositorio de ejercicios oficiales del lenguaje P4), GitHub. Disponible en: <https://github.com/p4lang/tutorials>
- [7] p4lang, *Behavioral Model (bmv2)*, GitHub. Disponible en: <https://github.com/p4lang/behavioral-model>